

Product Management REST API Implementation Technical Documentation

1. Executive Project Overview

This document serves as a complete validation record for the backend REST API development tasks completed. The objective was to design a clean, structured Node.js web application utilizing the Express web framework and a MongoDB non-relational database database tier. The architecture follows industry-standard MVC (Model-View-Controller) decoupling paradigms to optimize modularity, error recovery pipelines, and maintainability. Additionally, an advanced streaming file ingestion layer using Multer was successfully integrated to accommodate multi-part file uploads alongside standard JSON documents.

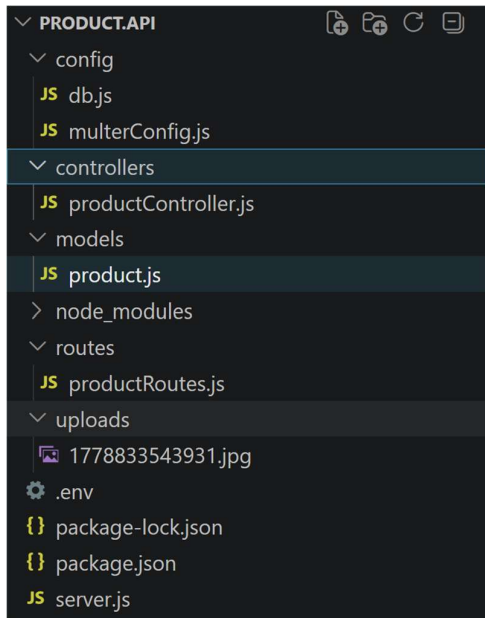
2. Engineering Technology Stack Reference

Component	Technology / Package	Core Functional Purpose Architecture
Runtime Environment	Node.js (v24.15.0)	Executes asynchronous server-side JavaScript events code pipeline.
Web Application Layer	Express.js Framework	Handles route mappings, intermediate middleware, HTTP transaction methods.
Database Modeling	MongoDB & Mongoose ORM	Manages database persistency with schema definitions and validations.
Multipart Parsing Engine	Multer Middleware	Intercepts inbound streams to write uploaded images safely to server disk.

3. Task 1: System Blueprint & Server Architecture Setup

The initial milestone required generating an isolated workspace environment named 'product-api' and creating a structured workspace layout designed to decouple application concerns into independent sub-modules

3.1 Core Architecture Layout Blueprint



3.2 Core Initialization Script (server.js)

```
JS server.js > ...
1  const express = require('express');
2  const dotenv = require('dotenv');
3  const connectDB = require('./config/db.js');
4  const productRoutes = require('./routes/productRoutes.js');
5
6  // Load environment configurations
7  dotenv.config();
8
9  // Connect to MongoDB instance
10 connectDB();
11
12 const app = express();
13
14 // Middleware to parse incoming raw JSON requests in req.body
15 app.use(express.json());
16 //so anyone can view image paths directly from a web browser window layout:
17 app.use('/uploads', express.static('uploads'));
18
19 // Base root testing route
20 app.get('/', (req, res) => {
21   res.send('Product API Server is running...');
22 });
23
24 // Link our product router path
25 app.use('/api/products', productRoutes);
26
27 const PORT = process.env.PORT || 5000;
28
29 app.listen(PORT, () => {
30   console.log(`Server running in development mode on port ${PORT}`);
31 });
```

```
part.js:36:13)
PS C:\Users\sreen\OneDrive\Pictures\Bharath Gitam\internship\task-5\product.api> npm run dev

> product.api@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: ◇ secrets for agents [www.dotenvx.com]
Server running in development mode on port 5000
MongoDB Connected: localhost
```

4. Task 2: Database Schema & Product Entry Ingestion API

A structural Mongoose definition model was constructed to safely validate product profiles. To accommodate the addition of visual records discussed during review sessions, the system was expanded with an automated file-upload parser.

4.1 Product Object Model Definition (models/Product.js)

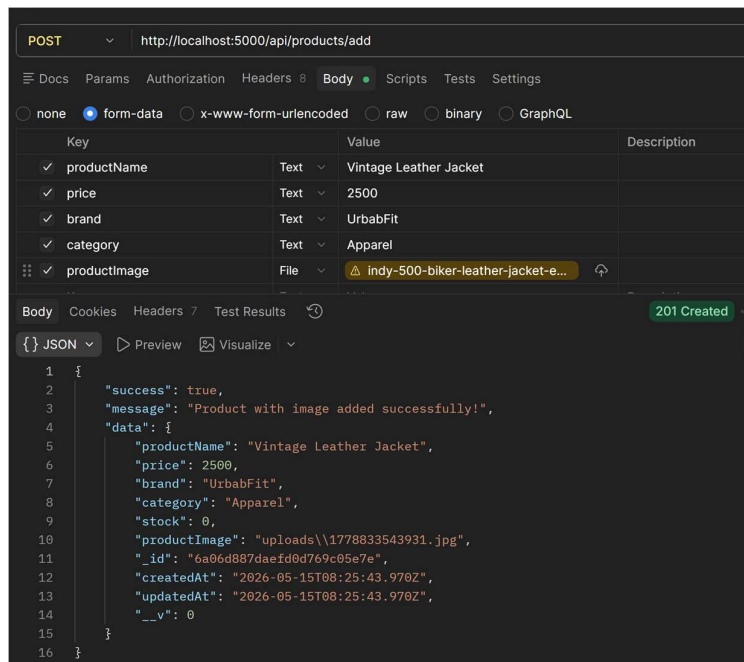
```
models > JS product.js > ...
1  const mongoose = require('mongoose');
2
3  // We declare it exactly ONCE here
4  const productSchema = new mongoose.Schema({
5    productName: {
6      type: String,
7      required: true
8    },
9    price: {
10     type: Number,
11     required: true
12   },
13   brand: {
14     type: String,
15     required: true
16   },
17   category: {
18     type: String,
19     required: true
20   },
21   stock: {
22     type: Number,
23     default: 0
24   },
25   productImage: {
26     type: String
27   }
28 }, { timestamps: true });
29
30 module.exports = mongoose.model('Product', productSchema);
```

4.2 Multi-part Ingestion Handling (controllers/productController.js)

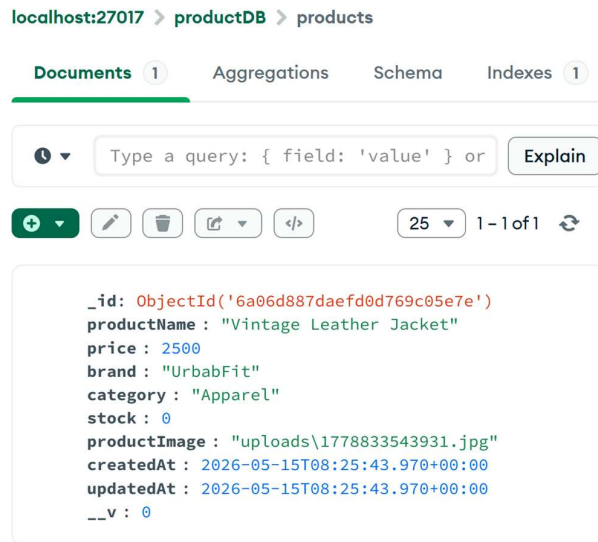
```
controllers > JS productController.js > ...
1  const Product = require('../models/Product');
2
3  // @desc   Add a new product (Updated to handle images!)
4  // @route   POST /api/products/add
5  const addProduct = async (req, res) => {
6    try {
7      // 1. Grab the normal text fields from the request body
8      const { productName, price, brand, category, stock } = req.body;
9
10     // 2. CHECK FOR THE IMAGE: Multer puts the uploaded file information inside 'req.file'.
11     // If a file exists, we grab its file path (e.g., "uploads/171575700123.png").
12     // If no file was uploaded, we just leave it as an empty string "".
13     const productImage = req.file ? req.file.path : "";
14
15     // 3. Create a new instance of the Product model, adding the image path to it
16     const newProduct = new Product({
17       productName,
18       price,
19       brand,
20       category,
21       stock,
22       productImage // <-- This passes the image path to our database entry
23     });
24
25     // 4. Save everything directly to MongoDB
26     const savedProduct = await newProduct.save();
27
28     res.status(201).json({
29       success: true,
30       message: "Product with image added successfully!",
31       data: savedProduct
32     });
33   } catch (error) {
34     res.status(500).json({
35       success: false,
36       message: "Server Error: Could not save product.",
37       error: error.message
38     });
39   }
40 };
41
```

```
41
42 // @desc   Get all products (This stays exactly the same!)
43 // @route   GET /api/products
44 const getProducts = async (req, res) => {
45   try {
46     const products = await Product.find({});
47     res.status(200).json({
48       success: true,
49       count: products.length,
50       data: products
51     });
52   } catch (error) {
53     res.status(500).json({
54       success: false,
55       message: "Server Error: Could not retrieve products.",
56       error: error.message
57     });
58   }
59 };
60
61 module.exports = {
62   addProduct,
63   getProducts
64 };
```

ADDING PRODUCTS:



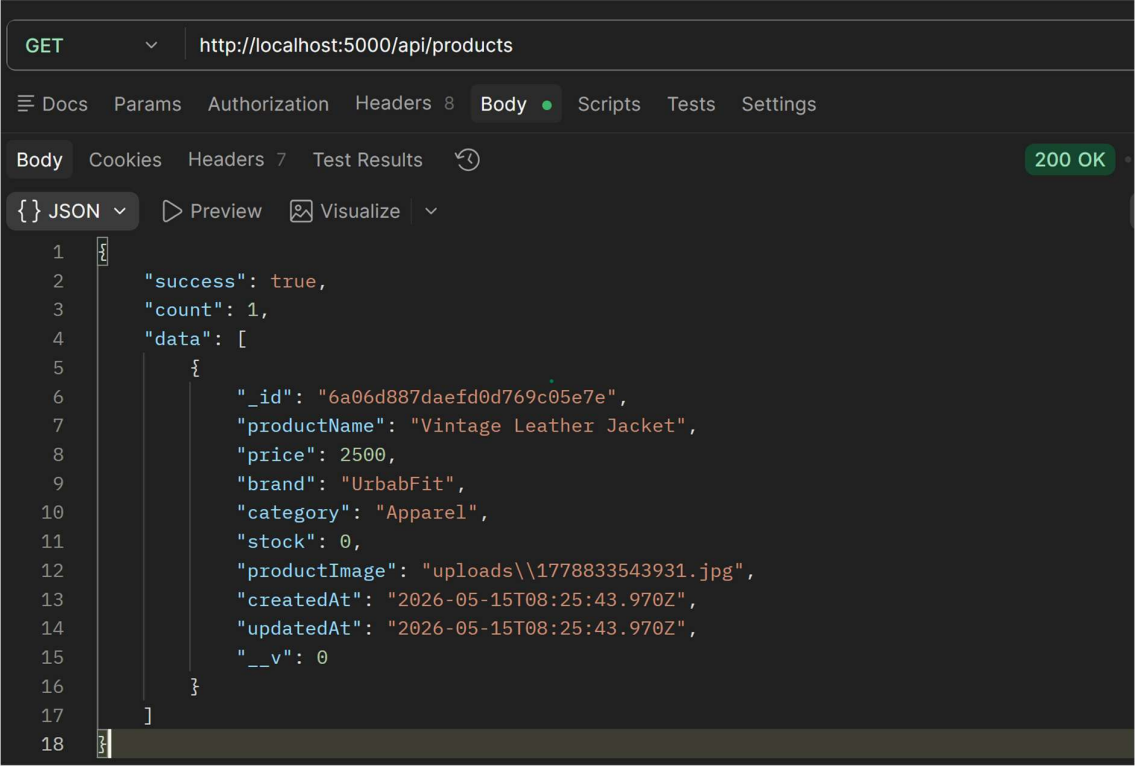
MongoDB Compass Collection view showing the product saved into the Database Room]



5. Task 3: Product Inventory Document Listing Retrieval API

Task 3 handles the recovery of stored items from our storage rooms. The data is pulled straight from the MongoDB cluster and pushed cleanly to the caller in structured formats.

5.1 Inventory List Query Engine Implementation



```
GET http://localhost:5000/api/products

Body
200 OK

{
  "success": true,
  "count": 1,
  "data": [
    {
      "_id": "6a06d887daefd0d769c05e7e",
      "productName": "Vintage Leather Jacket",
      "price": 2500,
      "brand": "UrbabFit",
      "category": "Apparel",
      "stock": 0,
      "productImage": "uploads\\1778833543931.jpg",
      "createdAt": "2026-05-15T08:25:43.970Z",
      "updatedAt": "2026-05-15T08:25:43.970Z",
      "__v": 0
    }
  ]
}
```